

Final Report: Model Extraction Attack

Stealing a Classifier via the Equation-Solving Attack

Tanzim Tousif - 2105071
Dibya Jyoti Sarkar - 2105084
Course: CSE-406 Cyber Security Sessional
Supervisor: Shattik Islam Rhythm
Lecturer
Department of CSE, BUET

1 Introduction

This project is about a black-box *model extraction* attack that reconstructs a deployed classifier from query access alone. Our target is a multiclass logistic-regression model served over a REST prediction API. Following the equation-solving attack of Tramèr et al. (USENIX Security 2016)[1], the adversary submits crafted inputs, reads the returned confidence vectors, and inverts the softmax to recover the victim's weights exactly. We further design an empirical study of how output-rounding defenses degrade extraction fidelity.

2 Experimental Setup

We used scikit-learn's built-in iris dataset to conduct our experiment. It has four features: sepal length, sepal width, petal length, petal width. All units are in centimeters. There are three species: *iris setosa*, *iris versicolor*, *iris virginica*. Attacker side sends queries to victim/defender server which is a black box model. Victim/defender side can send the probabilities of the input features belonging to each species. From that, attacker side uses "equation solving attack" to get the weights and biases of the model.

3 Attack Process

Figure 1 shows the end-to-end pipeline. The left column is the adversary (`extractor.py`), the middle column is the prediction service (`api.py`), and the right column is the victim process (`victim.py`), which holds the trained scaler and the softmax parameters $W \in \mathbb{R}^{c \times d}$, $b \in \mathbb{R}^c$ in memory and never serialises them over the wire. Everything the attacker learns crosses the dashed trust boundary as ordinary JSON.

The loop is deliberately cheap: steps 1–3 are repeated for n random probes, and only then does the attacker do any local computation. Steps 4–6 are pure linear algebra. No gradient descent, no training data, no labels are required at this stage. Choosing a fixed reference class r and taking log-ratios of the returned probabilities cancels the softmax normaliser, so each response contributes $c - 1$ exact linear equations in the unknown parameter differences $\theta_i = [(w_i - w_r)^\top, (b_i - b_r)]^\top$.

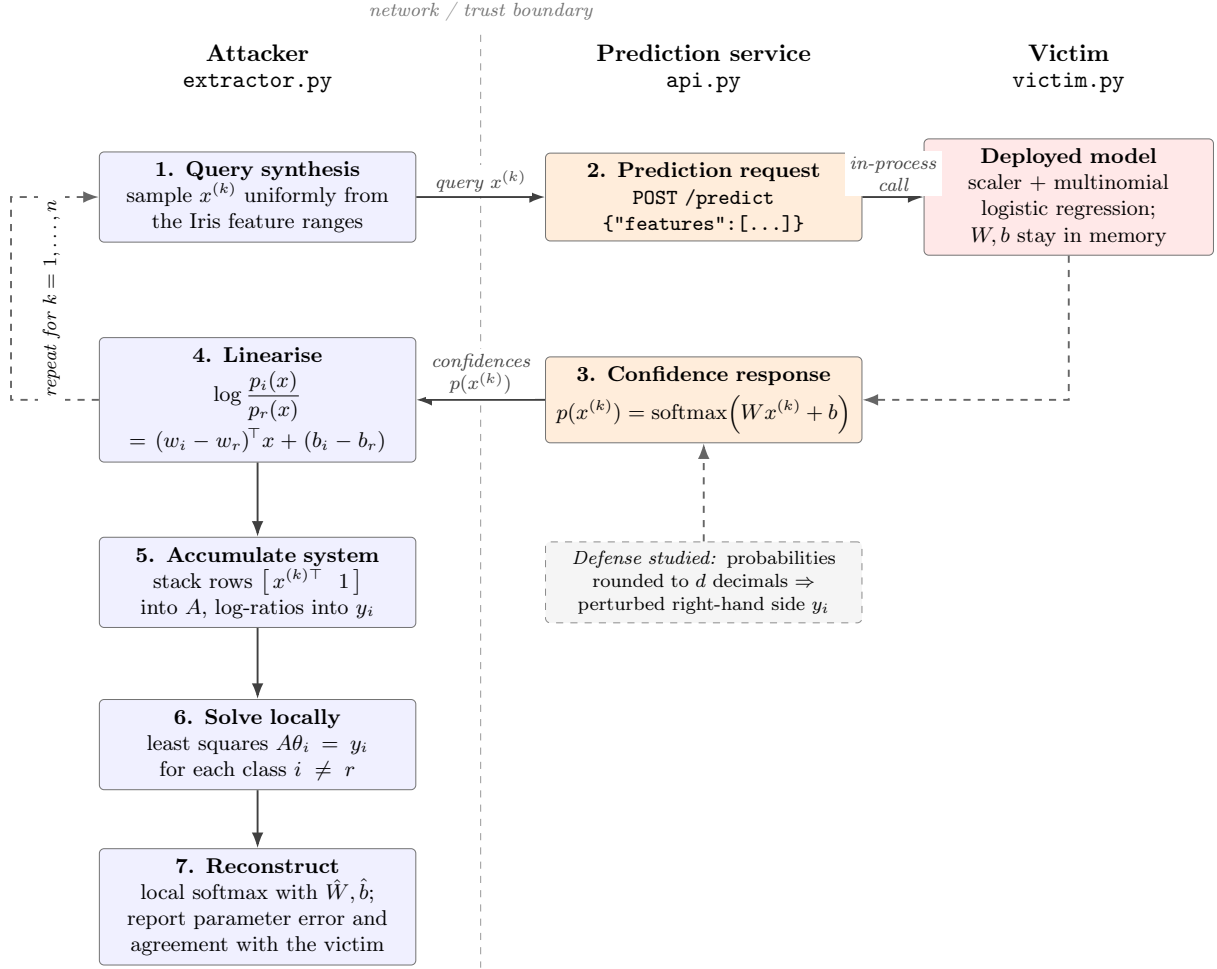


Figure 1: End-to-end view of the equation-solving extraction attack. Solid arrows carry the live query/response flow; dashed arrows carry derived or optional paths. The attacker never observes W or b : it observes only $p(x)$ at chosen points and recovers the parameter differences by solving a linear system offline.

4 Results

We have conducted the experiments varying number of queries and also conducted an experiment when defender uses countermeasures by rounding off decimal points by 2 place. The parameter error is calculated using below formula

$$\text{parameter_error} = \left\| \hat{W} - W \right\|_2 + \left\| \hat{b} - b \right\|$$

Table 1: Model Extraction and Fidelity Results across Varying Query Counts

Queries	Mean Abs. Prob. Error	Parameter Error	Victim Acc.	Attacker Acc.
50	3.74×10^{-16}	1.41×10^{-14}	1.0	1.0
100	3.16×10^{-16}	7.93×10^{-15}	1.0	1.0
500	8.76×10^{-16}	1.08×10^{-14}	1.0	1.0
1000	5.49×10^{-16}	1.54×10^{-14}	1.0	1.0

Table 2: Defender rounding upto 2 decimal points while sending output probabilities.

Queries	Mean Abs. Prob. Error	Parameter Error	Victim Acc.	Attacker Acc.
50	0.2546	365.67	1.0	0.640
100	0.1765	541.95	1.0	0.810
500	0.2112	525.21	1.0	0.722
1000	0.2031	511.36	1.0	0.736

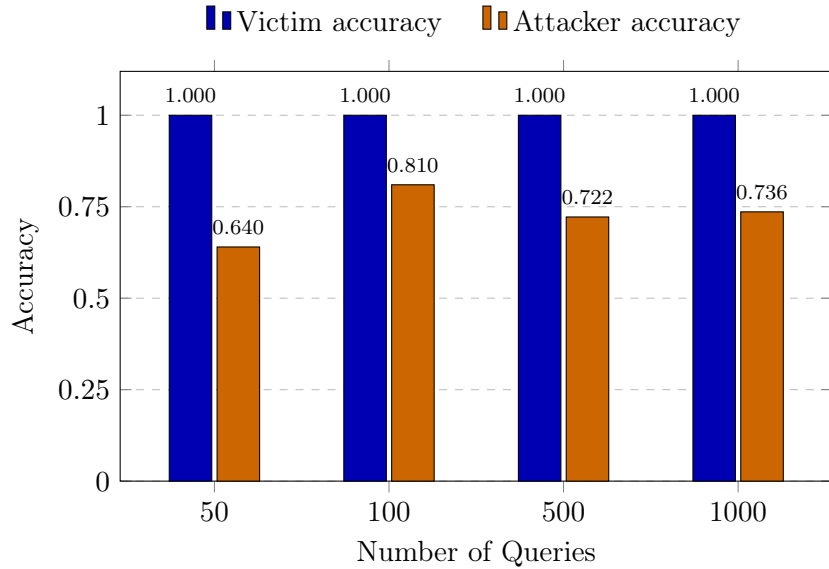


Figure 2: Defender rounding output probabilities to 2 decimal places.

5 Explanation

Table 1 shows that attacker successfully extracted model parameters near perfectly and also achieved same accuracy as victim/defender. The parameter extraction isn't fully perfect because

of randomization, non-linearity in model parameters. Figure 2 shows that victim/defender applied countermeasures by sending output probabilities upto 2 decimal places. As a result the accuracy of attacker side has dropped significantly. Additionally, Table 2 shows that parameter error is significantly higher due to defender’s countermeasures. However, one unique thing can be noticed. When query count is 100, parameter error is the highest but the accuracy is also highest. This validates that attacker’s extracted model might get higher accuracy despite not having the same parameters[3]

6 Conclusion

The project demonstrates model extraction attack via prediction APIs[1]. In order to prepare defense against these kind of attack, one solution is to round off the output probabilities so that attacker cannot use that to reverse engineer the model parameters. Additionally, it validates that higher accuracy is not always related to higher fidelity due to non-linearity and randomization.[3].

7 Contribution

- Tanzim Tousif (2105071): Implemented victim side API, tested the whole project.
- Dibya Jyoti Sarkar (2105084): Implemented attacker side, tested the whole project.

References

- [1] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, T. Ristenpart. *Stealing Machine Learning Models via Prediction APIs*. USENIX Security, 2016. arXiv:1609.02943.
- [2] T. Orekondy, B. Schiele, M. Fritz. *Knockoff Nets: Stealing Functionality of Black-Box Models*. CVPR, 2019. arXiv:1812.02766.
- [3] M. Jagielski, N. Carlini, D. Berthelot, A. Kurakin, N. Papernot. *High Accuracy and High Fidelity Extraction of Neural Networks*. USENIX Security, 2020. arXiv:1909.01838.